

HelixOTA

HelixOTA

Універсальныя, дэкапліраваныя абнаўленні праз паве́тра — бяспека ад брыку без кампрамісаў.

Кароткае змесціва

Helix OTA — універсальная сістэма абнаўленняў праз паве́тра (OTA) з глыбокім дэкаплінгам: Go-кіравальная плоскасць разам з кліенцкімі агентамі для кожнай АС, распрацаваная для бяспечнага, этапнага разгортвання абнаўленняў прашыўкі і праграм на паркі прылад ад адзінай платы да мільёнаў дэвайсаў. Першая мэта — Android 15 на Orange Pi 5 Max.

Кароткае апісанне

Helix OTA — універсальная сістэма абнаўленняў праз паве́тра, якая ўключае Go-кіравальную плоскасць і кліенцкія агенты для кожнай АС. Яна створана для забеспячэння нулявой пашкоджанасці сістэмы, правяраных загрузак і дэталізаванага этапнага разгортвання. Першай мэтай з’яўляецца Android 15 на Orange Pi 5 Max, запланаваны таксама адаптары для Linux і Windows.

Падрабязнае апісанне

Helix OTA — універсальная, гнуткая і глыбока дэкапліраваная сістэма абнаўленняў праз паве́тра (OTA), створаная на аснове аднаго нязменнага прынцыпу: абнаўленне ніколі не павінна ператвараць працуючы дэвайс у брык. Яна складаецца з Go-сервернай **кіравальнай плоскасці**, кліенцкіх **SDK/агентаў** для кожнай АС і **панэлі кіравання**, і распрацавана такім чынам, каб яе можна было ўбудоваць у *любую* аперацыйную сістэму праз зменныя адаптары, а не перапісваць з нуля для кожнай платформы. Першай мэтай з’яўляецца Android 15 (усе варыянты) на Orange Pi 5 Max, дзе канвеер зборкі генеруе

вобразы для прашыўкі разам з правэраным OTA-файлам .zip і абавязковымі файламі хэшаў, каб ніводзін артэфакт не трапіў на дэвайс без правэранай подпісы. Linux, Windows і іншыя АС чакаюць сваёй чаргі за тым жа адаптарным швом — без неабходнасці перапісваць усё зноўку.

Дызайн сістэмы заснаваны на жорсткіх гарантыях, абвешчаных апэратарам і разглядаемых як нязменныя архітэктурныя інварыянты: нулявая пашкоджанасць сістэмы, абавязковая праверка кожнага артэфакта перад разгортваннем, дэталізаванае разгортванне (адразу ці паэтапна: 5/10/30...100% з магчымасцю прыпынку і працягу), поўная назіральнасць за флотам прылад і лінейная маштабавальнасць ад адзінай платы на сталё да мільёнаў дэвайсаў у полі. Замкнутая архітэктурная спалучае натыйныя Android A/B-абнаўленні на ўзроўні прылады — AOSP update_engine з AVB/dm-verity і аўтаматычным адкатам пры няўдалым загрузванні — з уласнай, дэкапіраванай Go-кіравальнай плоскасцю, каб бяспека захоўвалася як на ўзроўні чыпаў і загрузкі, так і на серверы, а не ў адным крохкім пласце. Два швы наўмысна зроблены здымнымі: адаптарны шво для АС, які гарантуе сапраўдную універсальнасць, і шво рухавіка разгортвання, што робіць этапныя кампаніі незалежнымі ад АС. Уся сістэма падзелена на шэсць публічных, незалежна версіяваных ota-* падмадуляў — гэта перавыкарыстоўваемыя будаўнічыя блокі, а не маналіт.

Helix OTA цяпер знаходзіцца на этапе распрацоўкі спецыфікацый, даследаванняў і пашырэння тэставага пакрыцця. Рэпазіторый змяшчае аўтарытэтны корпус дызайну, сістэму экспарту дакументацыі і каркас падмадуляў. У адпаведнасці з прынцыпамі кіравання без падману, адкрыта заяўляецца, што гатовыя да вытворчасці сервер і агент пакуль не існуюць. На сённяшні дзень даступны толькі праект і яго каркас, і гэта адкрыта пазначана.

Змест

Чаму мы яго стварылі

OTA звычайна перавынаходзяць для кожнай прылады і кожнай АС, а няўдачнае абнаўленне можа «зацагляць» усю партыю. Helix OTA быў створаны як адзіная ўніверсальная сістэма абнаўленняў з прыярытэтам бяспекі, якую любая АС можа прыняць праз адаптары, з гарантыямі адкату і праверкі, убудаванымі ў архітэктурную, а не далучанымі постфактум.

Чаму гэта змяняе правілы гульні

Яна адмаўляецца разглядаць «ніколі не зацагляць прыладу» і «распаўсюджваць абнаўленні паступова і пад назіраннем» як найлепшыя практыкі, якія спадзяюцца вытрымаць нагрузку, — гэтыя прынцыпы з’яўляюцца архітэктурнымі інварыянтамі, убудаванымі як у шлях загрузкі, так і ў кіравальны слой. А дзякуючы таму, што рухавік распаўсюджвання і слой АС зроблены заменнымі швамі, а не жорстка замацаванымі здагадкамі, той жа кіравальны слой можа сёння кіраваць Android і быць гатовым кіраваць іншымі аперацыйнымі сістэмамі ў будучыні проста праз дадаванне адаптара — без разгалінавання, перапісу ці перавынаходства тых гарантый бяспекі, якім вы ўжо давяраеце.

Што новага

- **Два заменныя швы** — шво адаптара АС і рухавік распаўсюджвання, незалежны ад АС, — якія ператвараюць «універсальнасць» з маркетынгавага тэрміна ў структурную ўласцівасць кодавай базы.
- **Бяспека ў глыбіню:** на прыладзе — натыўны A/B Android (update_engine) + AVB/dm-verity + аўтаматычны адкат пры збоі загрузкі, якія накладваюцца *наверх* сервернай праверкі артэфактаў — абнаўленне павінна прайсці некалькі незалежных бар’ераў, перш чым яно зможа замацавацца.
- **Дэкампанаваны падыход з каталогам на першым месцы:** разбіццё на шэсць паўторна выкарыстоўваемых, незалежна версіянаваных падмадуляў ota-*, якія можна спажываць паасобку, замест таго каб глытаць маналіт.
- **Асноўны транспарт HTTP/3 (QUIC)** з аўтаматычным пераходам на HTTP/2 і дамоўленым сцісканнем Brotli/gzip — сучасная дастаўка з нізкай затрымкай, якая дэградацыйнае з грацыёзнасцю, а не правальваецца.
- **Антыблефная інжынерія:** дызайн і статус відавочна пазначаны як стадыя спецыфікацыі, і нішто непабудаванае ніколі не выдаецца за гатовае — шчырасць як інжынерная каштоўнасць першага парадку, а не адмаўленне ў зносах.

Найбольшыя тэхнічныя выклікі і як мы іх пераадолелі

- **Гарантаваць, што дрэннае абнаўленне ніколі не зацагляе прыладу** — найцяжэйшае абяцанне ў OTA. Вырашана шляхам абавязковага выкарыстання натыўнага A/B Android на прыладзе: update_engine запісвае ў неактыўны слот, у той час як актыўны слот працягвае

працаваць, AVB/dm-verity крыптаграфічна правярае ланцужок загрузкі, а калі новы слот не запускаецца, прылада аўтаматычна адключваецца — усё гэта падмацавана абавязковай папярэдняй праверкай артэфактаў, каб пашкоджаны груз быў знойдзены яшчэ да таго, як пакіне сервер.

- **Адна сістэма для многіх аперацыйных сістэм** — вырашана адмовай убудоваць здагадкі пра Android у ядро. Зменны шво адаптара АС ізалюе спецыфіку платформы, а рухавік распаўсюджвання, незалежны ад АС, захоўвае лагіку кампаній пераноснай, прычым кожны з іх зроблены асобным падмадулем, каб дадаванне новай АС было дадаткам, а не хірургічным умяшаннем ва ўсё цэлае.
- **Паэтапныя, прыпыняльныя распаўсюджванні** — вырашана з дапамогай асобнага рухавіка распаўсюджвання, які аперае працэнтнымі кагортамі з парогамі поспеху/памылак і відавочным кіраваннем прыпынку/прасоўвання, наўмысна адлучаным ад HTTP, каб той жа рухавік мог кіраваць кампаніямі незалежна ад транспарту.

Змест

Тэхналагічны стэк

- **Go + Gin** — абраны дзякуючы мадэлі канкурэнтнасці і кампактнаму разгортванню; забяспечвае работу кантрольнай плоскасці, рухавіка разгортвання і валідатараў артэфактаў, прадастаўляючы асноўны інтэрфейс REST /api/v1.
- **Kotlin/KMP** — абраны для таго, каб агенту OTA на прыладзе з Android можна было выкарыстоўваць адзін і той жа код на розных мэтавых платформах; кантралюе поўны цыкл працы з прыладай: апытанне / спампоўванне / праверка / ужыванне / справаздача.
- **HTTP/3 (QUIC) → HTTP/2** — QUIC абраны як асноўны транспарт для перадачы з нізкай затрымкай і ўстойлівасцю на мабільных каналах з стратамі, з аўтаматычным пераключэннем на HTTP/2, каб ніводная прылада не засталася без падтрымкі; **Brotli/gzip** перагаворваюцца для кожнага запыту, каб памяншаць аб'ём перадаваных даных.
- **PostgreSQL** — абраны для забеспячэння рэляцыйнай цэласнасці паміж рэестрам прылад, кампаніямі і тэлеметрыяй, дзе карэктнасць стану парку важнейшая за хуткасць запісу.
- **MinIO / S3** — абраны як сховішча артэфактаў, каб вялікія вобразы прашывак захоўваліся ў стандартным аб'ектным сховішчы, аддзеленым ад рэляцыйнага ўзроўню.
- **AOSP update_engine + AVB/dm-verity + boot_control** — абраны таму, што выкарыстанне ўласных, правяраных на

практыцы механізмаў Android Virtual A/B і правэранай загрузкі бяспечней, чым стварэнне ўласнай сістэмы абнаўлення; выкарыстоўваецца для кіравання пераключэннем слотаў і крыптаграфічнай праверкай загрузкі на прыладзе.

- **React** — абраны для панэлі кіравання, дзе аператары ўваходзяць у сістэму, загружаюць артэфакты, кіруюць разгортваннем і сочаць за станам парку прылад у адным месцы.
- **OpenTelemetry + Prometheus/Grafana** — абраны для вендарнезалежнага маніторынгу; выкарыстоўваецца для таго, каб кожны этап разгортвання быў бачны ў метрыках і панэлях маніторынгу, а не заставаўся на ўзроўні здагадак.

Стан і адкрытасць інфармацыі

- **Стан: у распрацоўцы.** Згодна з прынцыпамі праекта па барацьбе з неабгрунтаванымі заявамі, **няма працоўнага сервера ці агента для прадукцыйнага асяроддзя** — гэта этап распрацоўкі спецыфікацый, даследаванняў і пашырэння пакрыцця тэстамі. Рэпазіторый змяшчае аўтарытэтны корпус дызайну, сістэму экспарту дакументацыі і структуру субмадуляў.
- Шэсць публічных паўторна выкарыстоўваемых субмадуляў (ota-protocol, ota-artifact-validator, ota-rollout-engine, ota-update-engine-bridge, ota-android-agent, ota-telemetry-schema) знаходзяцца ў github.com/HelixDevelopment/.
- Паказчыкі пакрыцця тэстамі і затрымкі ў рэпазіторыі — гэта ўнутраны рэестр праекта на бягучы момант, не пацверджаны незалежна. Нумары пунктаў HelixConstitution, на якія спасылаецца README, **НЕ ПРАВЕРАНЫ**.
- Ліцэнзія: **Apache-2.0**.

Прыярытэтны ўзровень: Helix-першасны.